Report

Remote Code Execution (RCE) and Command Injection

Suyash A Gunjal

Remote Code Execution

What is Remote Code Execution?

Remote code execution (RCE) is a vulnerability that lets a malicious hacker execute arbitrary code in the programming language in which the developer wrote that application. The term remote means that the attacker can do that from a location different than the system running the application.

Remote code execution is also known as code injection and remote code evaluation.

Prevention includes strong input validation, avoiding dangerous functions, applying least privilege, and regularly updating software.

Because of its impact and ease of exploitation, RCE is considered one of the most dangerous cybersecurity vulnerabilities.

How remote code execution (RCE) attacks work:

- **Injection vulnerabilities:** An injection vulnerability — such as SQL injection or command injection — is enabled by poor input sanitization. If a user provides a carefully crafted, malicious input, some of their provided data will be interpreted as commands to be run. This allows the attacker to force the vulnerable system to execute attacker-provided code.
- **Insecure deserialization:** Serialization simplifies the transmission of sets of data by packing it into a single string of bits to be unpacked by the recipient system. However, if the structure of serialized data is not well defined, an attacker may be able to craft an input that is misinterpreted when it is unpacked. Depending on how the data is stored and processed, this misinterpretation may allow the attacker to achieve code execution.
- **Out-of-bounds write:** A buffer is a fixed-size piece of memory that is allocated to store data. Insecure data reads or writes could allow an attacker to place data where it would be interpreted as code or as important control flow information for the application.

- **File management:** Some applications allow users to upload files to a server. The access that this provides may allow an attacker to upload a file containing malicious code and trick the application into executing it.

How to prevent remote code execution

Remote code execution attacks are made possible by vulnerabilities in corporate web applications. Some of the methods companies can use to reduce their exposure to RCE attacks include the following:

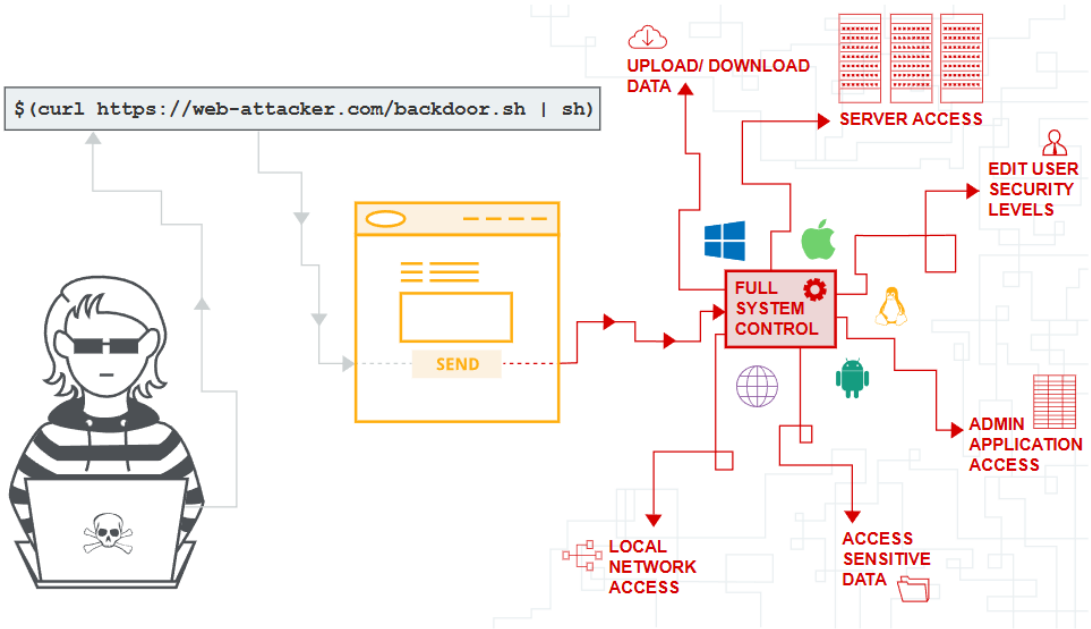
- **Vulnerability scanning:** A combination of static code analysis — to identify well-known vulnerability patterns — and dynamic analysis like fuzzing can help developers identify and correct RCE vulnerabilities before they can be exploited.
- **Updating and patching:** RCE vulnerabilities may be discovered in an application or the third-party libraries that it depends on. Promptly applying updates and patches reduces the window during which vulnerable code could be exploited.
- **Input validation:** Most RCE vulnerabilities involve poor input validation, where an application makes assumptions about user-provided data. Robust input validation can eliminate the threat of most RCE vulnerabilities.
- **Network monitoring:** Attempted RCE exploits can be detected and blocked by a web application and API protection (WAAP) solution. Deploying network security solutions in front of web applications can reduce the potential risks of unpatched vulnerabilities.

Remote Code Execution is a high-severity vulnerability that allows attackers to run malicious code on a remote system. Preventing RCE requires secure coding practices, strict input handling, updated software, and proper system configuration.

Command Injection

What is command injection?

- **Command Injection** in an API occurs when user input is executed as a system command rather than treated as data.
- This happens when the backend API passes unvalidated input into functions like exec(), system(), or command-line utilities.
- If exploited, an attacker can run unauthorized system commands, leading to data theft, malware installation, or full server control.
- Prevention includes strict input validation, parameterized execution, avoiding unsafe command functions, and applying least-privilege permissions.
- Because of its high impact and exploitability, command injection is considered a critical security risk in API development.



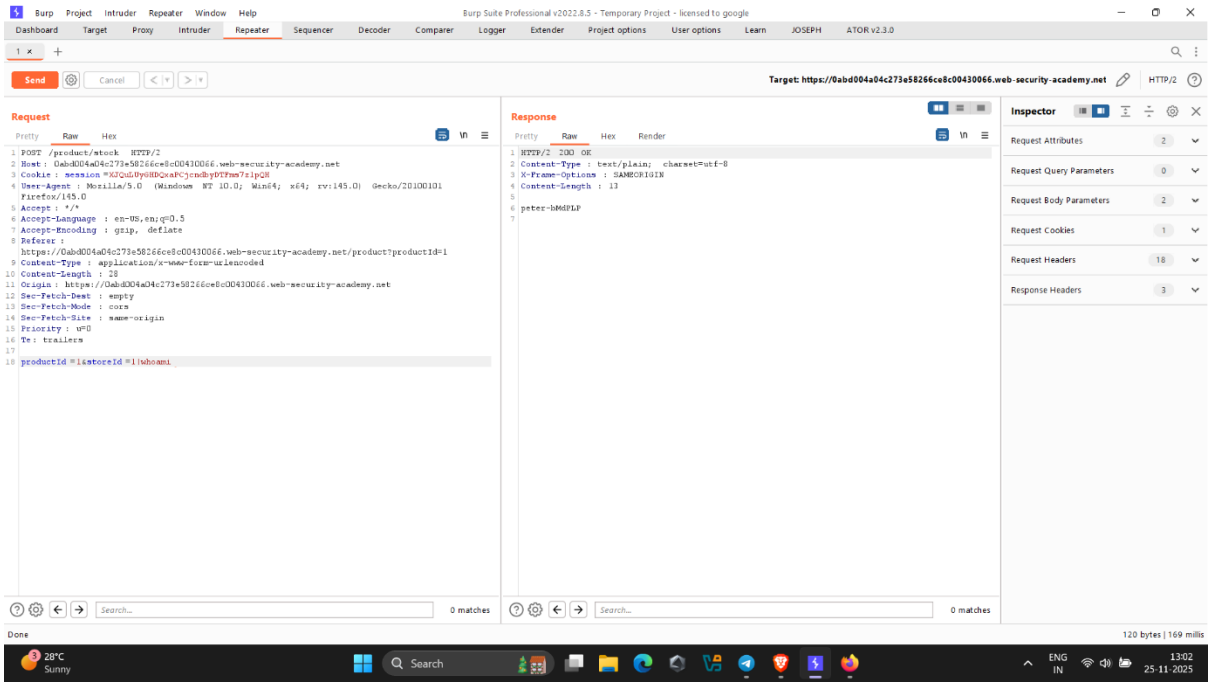
How Command Injection Works

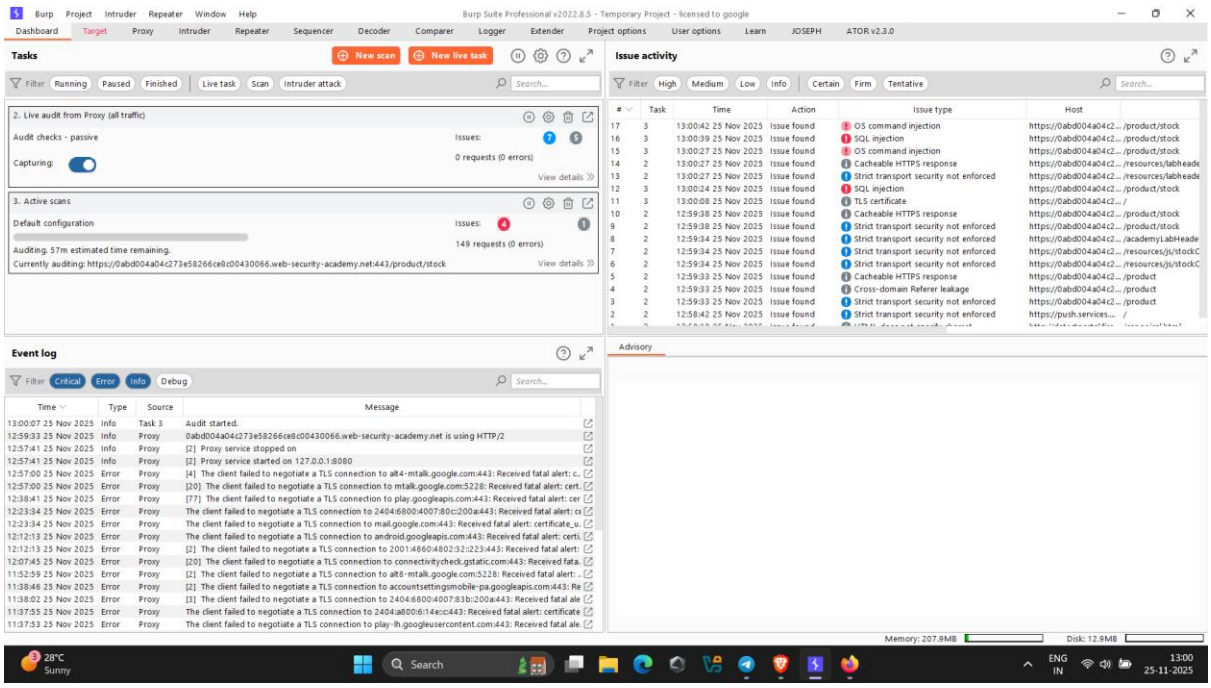
- Command injection happens when an application/API takes user input and sends it directly to the system shell (like Linux or Windows commands).
- Attackers modify the input to add extra commands (e.g., ; rm -rf / or && cat /etc/passwd).
- When the server executes that input, the attacker gains the ability to run system commands on the backend.
- This can lead to data exposure, privilege escalation, or full system compromise.

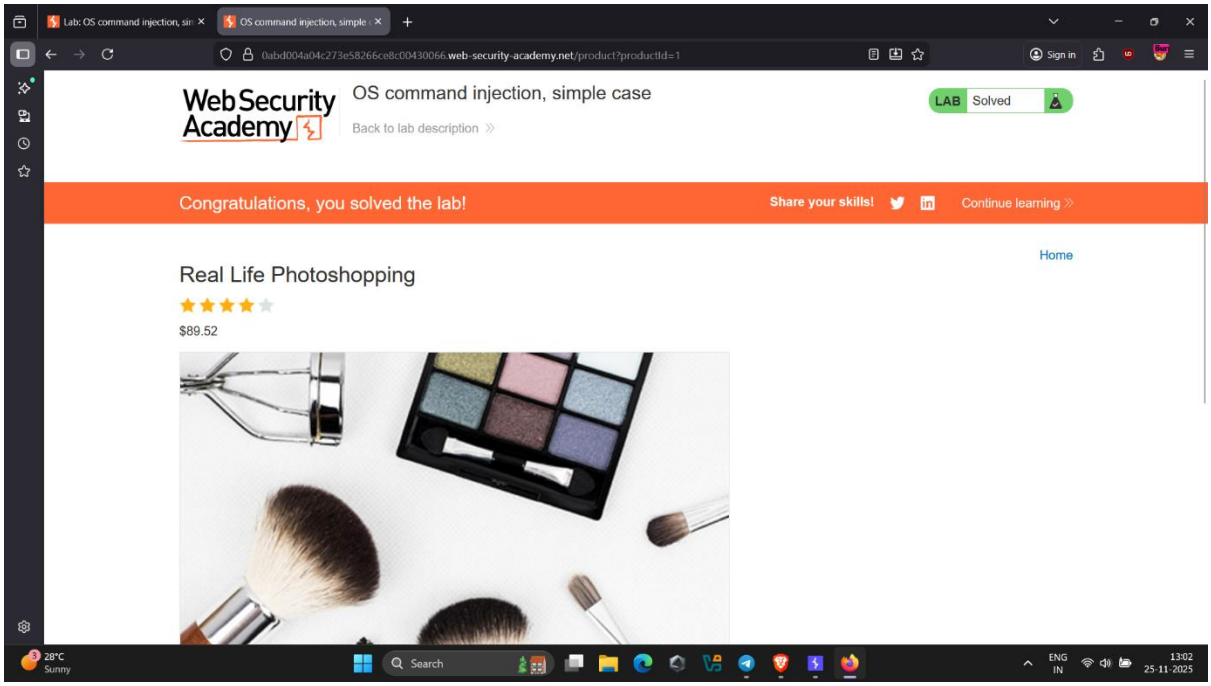
How to Prevent It

- Validate and sanitize all input using allowlists instead of relying on filtering.
- Avoid executing system commands directly; use safe APIs or parameterized functions.
- Apply least privilege to the application so even if exploited, damage is limited.
- Regularly update dependencies and use security tools or scans to detect vulnerabilities.

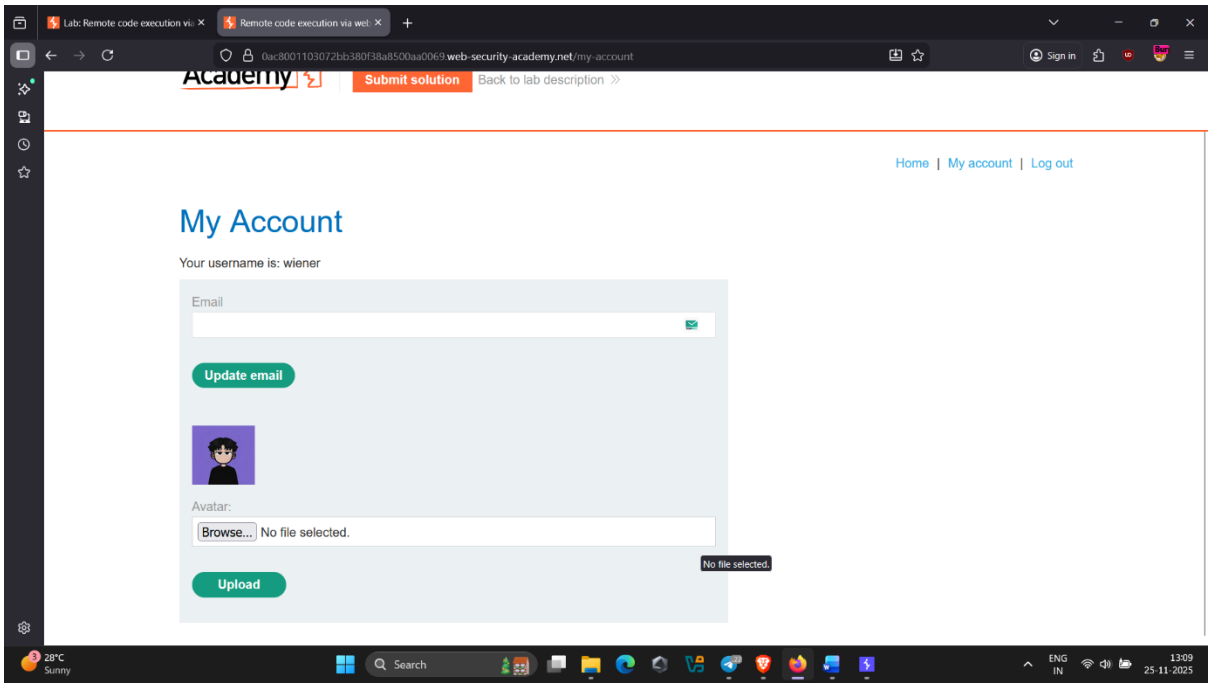
LAB: OS command injection, simple case:

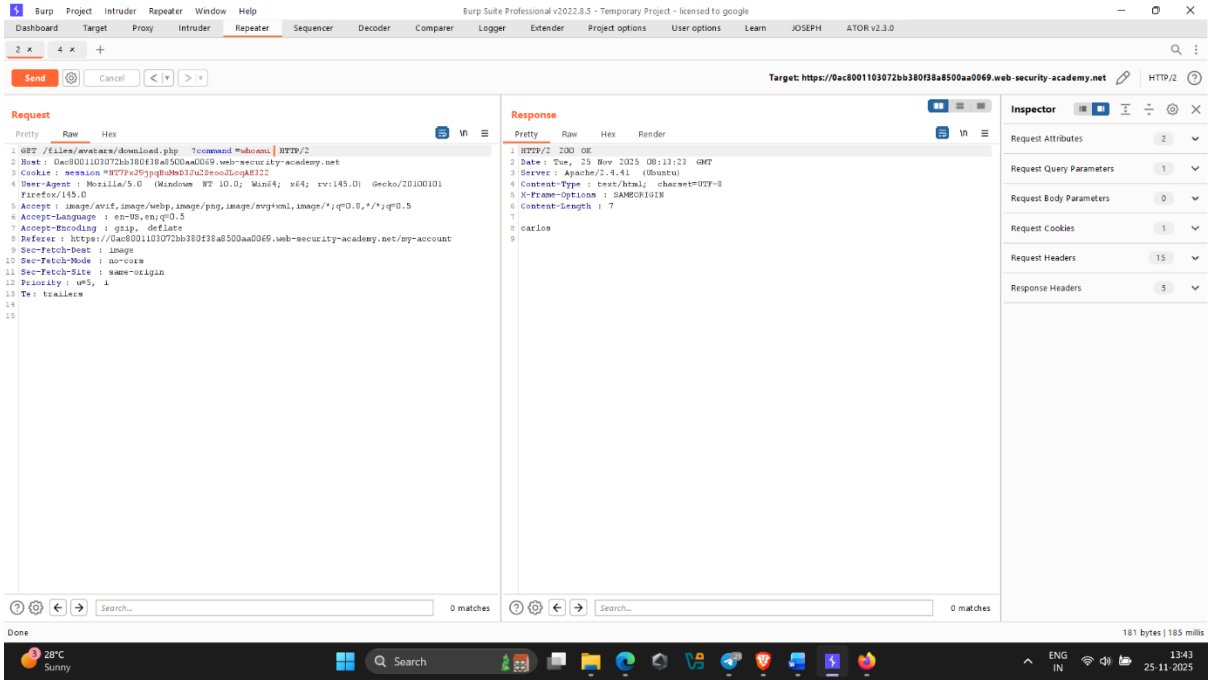


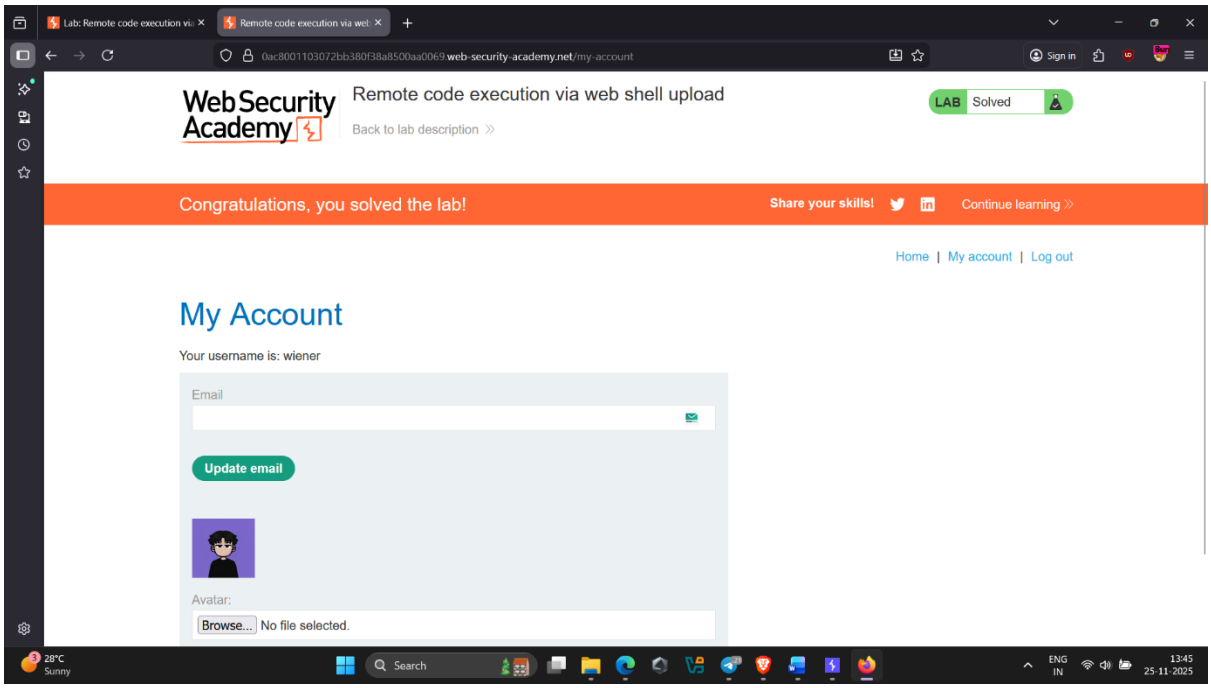




Lab: Remote code execution via web shell upload







References:

https://portswigger.net/web-security/file-upload/lab-file-upload-remote-code-execution-via-web-shell-upload

https://www.cloudflare.com/learning/security/what-is-remote-code-execution/